

What Can C++26 Reflection Do for You?

A Practical Experiment in Using Reflection
With an Emphasis on Binary Protocol Support

ANDY WEBBER

About me

Andy Webber

Retail Equity Trading Team Lead

andy.webber@sig.com

- First started programming with C++ in 1998 – Hello World!
- Joined Susquehanna in 2008
 - Order routing
 - Market data
 - Low-latency trading frameworks
 - Trading strategies
 - Proprietary and retail equity trading



Susquehanna is a global quantitative trading firm powered by scientific rigor, curiosity, and innovation. Our researchers, engineers, and traders work together to solve complex problems and build the high-performance systems that support our work across global markets.

We are hiring C++ developers globally at Susquehanna!

- Make a direct impact on the systems supporting our trading desks
- See new ideas and functionality move into production through short development cycles
- Investigate and propose technologies that advance our engineering capabilities
- Develop solutions optimized for performance, stability, and scale

<https://careers.sig.com/technology/jobs>



The Goals

- C++ as networking protocol technical spec
- Use reflection to generate a full parser
- Portable code
- Put reflection through a real-world trading use case
- Simple code as much as possible
- Illustrate modern compile time techniques
- gcc 16.1

What we'll cover

1

A wire protocol primer

packing, alignment, and the initial messages

2

Discovering reflection

std::meta, template for, and the core vocabulary

3

Identifying messages

finding the message structs by their shape

4

Data vs. Storage

the wire layout vs. the in-memory model

5

Parsing fields with conversions

the field loop and the conversion overload set

6

Annotations

carrying policy on the struct

A wire protocol primer

Structs on a wire - order book protocol

```
#pragma pack(push, 1) // struct layout == wire
namespace OrderProtocol {

    enum class MessageTypeId : uint8_t {
        order_add_message = 1,
        order_remove_message = 2,
        trade = 3 };

    enum class OrderSide : uint8_t {
        buy = 0, sell = 1 };

    struct PacketHeader {
        uint16_t data_length;
        uint32_t sequence_number;
    };

    struct MessageHeader {
        uint8_t message_type;
    };
};
```

```
struct OrderAdd : MessageHeader {
    static constexpr MessageTypeId message_type_id =
        MessageTypeId::order_add_message;
    uint32_t stock_id;
    uint32_t order_id;
    uint8_t side;
    uint32_t quantity;
    uint32_t price;
};

struct OrderRemove : MessageHeader {
    uint32_t order_id;
};

struct Trade : MessageHeader {
    uint32_t order_id;
    uint32_t quantity;
    uint32_t price;
};
}
#pragma pack(pop)
```

One message, byte by byte

OrderAdd - little-endian, no padding

offset	size	field	part
0	1	message_type (uint8)	MessageHeader
1	4	stock_id (uint32)	
5	4	order_id (uint32)	
9	1	side (uint8)	
10	4	quantity (uint32)	
14	4	price (uint32)	
			total 18

sizes: MessageHeader=1 OrderAdd=18 OrderRemove=5 Trade=13

Hazards: reading through the blob

```
// tempting: overlay a struct on the raw bytes
auto const msg = reinterpret_cast<OrderAdd const>(
    buffer.data() + offset);
uint32_t px = msg->price;    // unaligned load + aliasing violation
```

```
// C++23
auto const msg = std::start_lifetime_as<OrderAdd>(
    buffer.data() + offset);
uint32_t px = msg->price;    // unaligned load + not fully available
```

```
// what the parser does: copy into a correctly-aligned local
uint32_t sequence_number;
std::memcpy(&sequence_number, bytes.data(), sizeof(sequence_number));
// alignment satisfied, no aliasing UB
```

Exchange Protocol Examples

- NASDAQ ITCH
- NYSE Pillar
- Cboe PITCH
- 24X/LTSE MEMOIR
- IEX DEEP
- MIAX Pearl DoM
- Consolidated CTA/UTP
- Hundreds more...

Discovering reflection

std::meta::info

- #include <meta>
- Opaque type
- Copyable, equality comparable, no ordering
- consteval only
- std::meta::info{} // the (null) empty reflection
- Represents
 - Types, aliases, functions, variables, members, templates, namespaces, constant expression objects and values, ...
- namespace meta = std::meta;

Reflection and splicing

```
constexpr meta::info r1 = ^^int;

struct MyStruct { int i; float j; };
constexpr meta::info r2 = ^^MyStruct;
constexpr meta::info r3 = ^^MyStruct::i;

namespace NS { template<typename T> void DoStuff() {}; }
constexpr meta::info r4 = ^^NS;
constexpr meta::info r5 = ^^NS::DoStuff;
constexpr meta::info r6 = ^^NS::DoStuff<int>;

[:r1:] i = 5;
[:r2:] s{1, 2.0};
s.[:r3:] = 2;

[:r4:]::DoStuff<int>();
template [:r5:]<int>();
[:r6:]();
```

Asking what an info actually is

```
// "is it a ...?" predicates
is_type(i);           is_class_type(i);   is_enum_type(i);
is_namespace(i);      is_function(i);     is_array_type(i);
is_static_member(i);  is_nonstatic_data_member(i);

// "does it have a name, and what is it?"
has_identifier(i);     // is it named?
identifier_of(i);      // "message_type_id" (a string_view)

// turn one info into related infos / plain facts
type_of(i);            // the info of i's type
parent_of(i);          // the enclosing scope
size_of(i);            // sizeof, in bytes
remove_extent(i);      // element type of an array
```

First steps: enumerate a namespace

```
template<meta::info I>
void print_member_strings()
{
    constexpr auto ctx = meta::access_context::unchecked();

    template for (constexpr auto r :
        define_static_array(meta::members_of(I, ctx)))
    {
        std::println("{} ", meta::display_string_of(r));
    }
}

// call site
print_member_strings<^^OrderProtocol>();
```

meta::members_of: what's inside an entity

```
constexpr std::vector<meta::info>
    meta::members_of(meta::info entity, meta::access_context ctx);

// every member comes back as another meta::info
for (meta::info m : members_of(^^OrderProtocol, ctx))
    // m might be a struct, an enum, a namespace, a function...

// more specific queries in the same family:
nonstatic_data_members_of(^^OrderAdd, ctx);    // the fields
static_data_members_of(^^OrderAdd, ctx);       // e.g. message_type_id
bases_of(^^OrderAdd, ctx);                     // base classes?
enumerators_of(^^MessageTypeId);               // enum values
```

meta::access_context: whose view are you reflecting?

```
// most member queries take an access_context
members_of(^^OrderAdd, ctx);

// default access - public api:
constexpr auto ctx = meta::access_context::unprivileged();

// ignore access control - see all:
constexpr auto ctx = meta::access_context::unchecked();

// as the current code would see it (respects public / private):
constexpr auto ctx = meta::access_context::current();
```

meta::display_string_of: a name you can print

```
constexpr std::string_view
    meta::display_string_of(meta::info entity);

std::println("{} ", display_string_of(^OrderAdd));
// -> "OrderAdd" (implementation-defined, for humans)

// contrast with identifier_of - the exact source spelling:
identifier_of(field); // "message_type_id" (string_view)
has_identifier(field); // false for unnamed entities

// debugging:
std::println("{}::{}",
    display_string_of(parent_of(^MessageDataType)),
    display_string_of(^MessageDataType));
```

Reflection – error handling

```
static_assert(members_of(^int, ctx).size() == 0);
               ^~~~~~
error: non-constant condition for static assertion
error: uncaught exception of type 'std::meta::exception'

// constexpr vector<info>
//     members_of(info r, access_context ctx);
// Throws: meta::exception unless dealias(r) is a reflection
// representing either a class type that is complete from some
// point in the evaluation context or a namespace.
```

Caution: a consteval vector can't escape to runtime

```
constexpr std::vector<std::string_view>
member_strings(meta::info ns)
{
    constexpr auto ctx = meta::access_context::unchecked();
    std::vector<std::string_view> names;

    for (auto m : meta::members_of(ns, ctx))
        names.push_back(meta::display_string_of(m));

    return names;
}

constexpr auto names = member_strings(^OrderProtocol);

error: member_strings(^OrderProtocol) is not a constant expression
because it refers to a result of 'operator new'
```

template for: a compile-time loop that expands

```
// an expansion statement - unrolled at compile time
template for (constexpr auto member :
    define_static_array(meta::members_of(I, ctx)))
{
    // 'member' is a distinct constant each iteration,
    // so you can splice it or use it as a template argument:
    std::println("{} ", display_string_of(member));
    parse_field<[: type_of(member) :]>(bytes);
}

// a normal for cannot do this - each step needs a constant:
//     [: member :]      splice requires a constant expression
```

Protocol Parser Generator

Parser Generator Feature Choices

- Parse any protocol meeting requirements
- Copy raw data into strongly typed message storage
- Different storage per use-case
- Callback interface for message handling
- Optionally skip messages and fields
- Extensible type conversion
- Use annotations to declare protocol properties

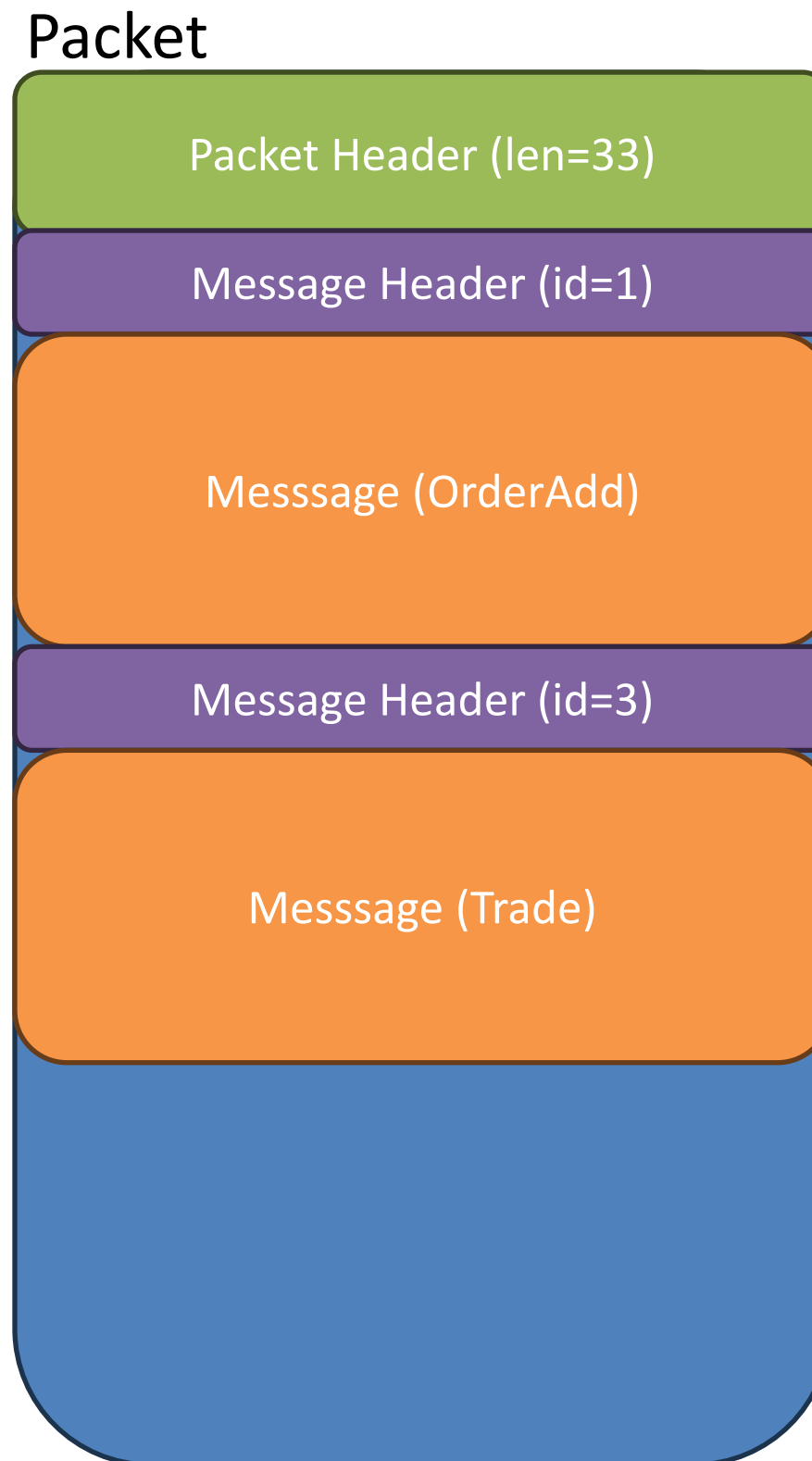
Parser Generator Interface Choices

- Namespaces to contain data and storage types
- Identifying message types
- Matching message type and field names
- Extension point for conversion

Two namespaces: wire format vs. app model

```
namespace Data {                                // namespace Storage {
    #pragma pack(push, 1)                       //
    struct MessageHeader {                      // struct MessageHeader {
        uint8_t message_type;                  //     MessageTypeId message_type;
    };                                          // };
                                                //
    struct OrderAdd : MessageHeader {           // struct OrderAdd {
        uint32_t stock_id;                     //     StockId    stock_id;
        uint32_t order_id;                     //     OrderId    order_id;
        uint8_t side;                          //     Shares     quantity;
        uint32_t quantity;                     //     Price      price;
        uint32_t price;                        //     Side       side;
        static constexpr MessageTypeId
            message_type_id =
            MessageTypeId::order_add_message;
    };                                          // };
}                                              // }
```

Multicast Packet example



Callback model

```
class MyMessageHandler
{
public:
    // business logic here
    void handle(OrderProtocol::Storage::PacketHeader&& message)
    {
    }
    void handle(OrderProtocol::Storage::MessageHeader&& message)
    {
    }
    void handle(OrderProtocol::Storage::OrderAdd&& message)
    {
    }
    void handle(OrderProtocol::Storage::OrderRemove&& message)
    {
    }
    void handle(OrderProtocol::Storage::Trade&& message)
    {
    }
};
```

Constructing the parser

```
template<meta::info ProtocolNs, typename CallbackObject>
struct Parser
{
    ...

    CallbackObject& handler;
}

template<meta::info ProtocolNs, typename CallbackObject>
auto get_protocol_parser(CallbackObject& handler)
{
    return Parser<ProtocolNs, CallbackObject>{handler};
}

auto parser = protocol_generator::get_protocol_parser<^^OrderProtocol>(cb);
```

parse_buffer: provide packet data, find all messages

```
void parse_buffer(std::span<std::byte const> buffer)
{
    constexpr auto ctx = meta::access_context::unchecked();
    constexpr auto DataNs = nested_namespace(ProtocolNs, "Data");
    constexpr auto StorageNs = nested_namespace(ProtocolNs, "Storage");

    // find only the real message types
    constexpr auto message_data_types =
        std::define_static_array(meta::members_of(DataNs, ctx) |
                                std::ranges::views::filter(is_message_type));

    // all derive from the same header?
    static_assert(check_common_base(message_data_types));

    ...
}
```

Finding the pieces: namespaces by name

```
constexpr meta::info
nested_namespace(meta::info outer, std::string_view name)
{
    auto ctx = meta::access_context::unchecked();
    for (meta::info m : meta::members_of(outer, ctx))
        if (meta::is_namespace(m) &&
            meta::has_identifier(m) &&
            meta::identifier_of(m) == name)
            return m;
    throw "not found";           // constexpr throw = compile error
}
```

Which structs are messages? Ask their members

```
constexpr bool is_message_type(meta::info info)
{
    constexpr auto ctx =
        meta::access_context::unprivileged();

    if (!meta::is_class_type(info))
        return false;

    for (meta::info field :
        meta::static_data_members_of(info, ctx))
        if (meta::identifier_of(field) == "message_type_id")
            return true;

    return false;
}

// ... members_of(Ns) | std::views::filter(is_message_type)
```

Caution: bases_of returns direct base class relationships, not types

```
constexpr bool check_common_base(std::span<meta::info const> message_infos)
{
    constexpr auto ctx = meta::access_context::unchecked();

    // assuming the first listed base is the header
    auto first_base = [ctx](meta::info message_info) {
        return meta::bases_of(message_info, ctx)[0]; };

    // compare all messages base
    return std::ranges::all_of(message_infos |
        std::ranges::views::transform(first_base),
        [&](auto base){ return base == bases.front(); });
}
```

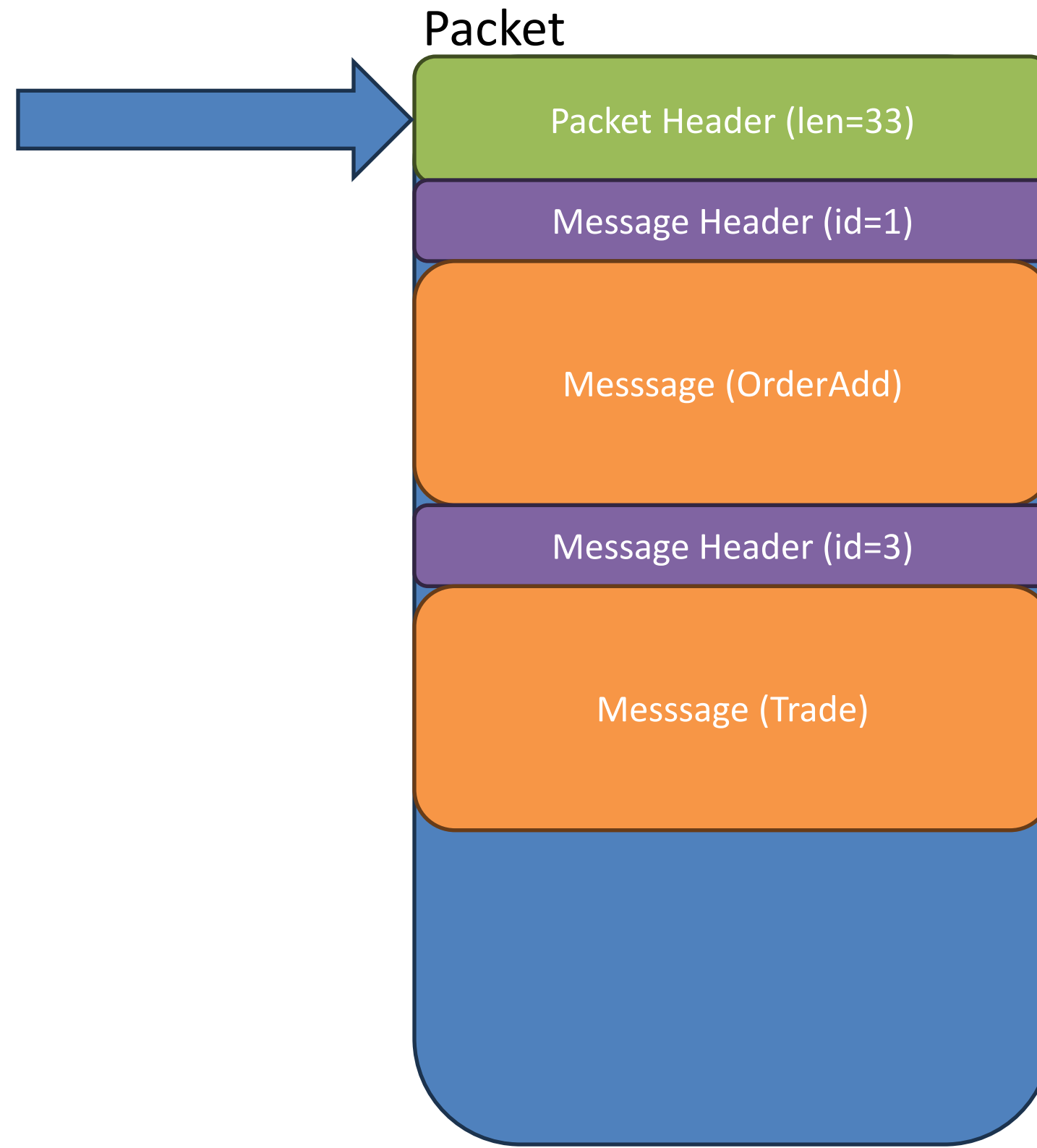
Caution: bases_of returns direct base class relationships, not types

```
constexpr bool check_common_base(std::span<meta::info const> message_infos)
{
    constexpr auto ctx = meta::access_context::unchecked();

    // assuming the first listed base is the header
    auto first_base = [ctx](meta::info message_info) {
        // needs type_of since bases_of returns base-derived relationships
        return meta::type_of(meta::bases_of(message_info, ctx)[0]); };

    // compare all messages base
    return std::ranges::all_of(message_infos |
        std::ranges::views::transform(first_base),
        [&](auto base){ return base == bases.front(); });
}
```

Multicast Packet example



parse_buffer: extract packet header

```
void parse_buffer(std::span<std::byte const> buffer)
{
    ...
    constexpr auto packet_header_data_info = find_struct(DataNs, "PacketHeader");
    constexpr auto packet_header_storage_info = find_struct(StorageNs, "PacketHeader");

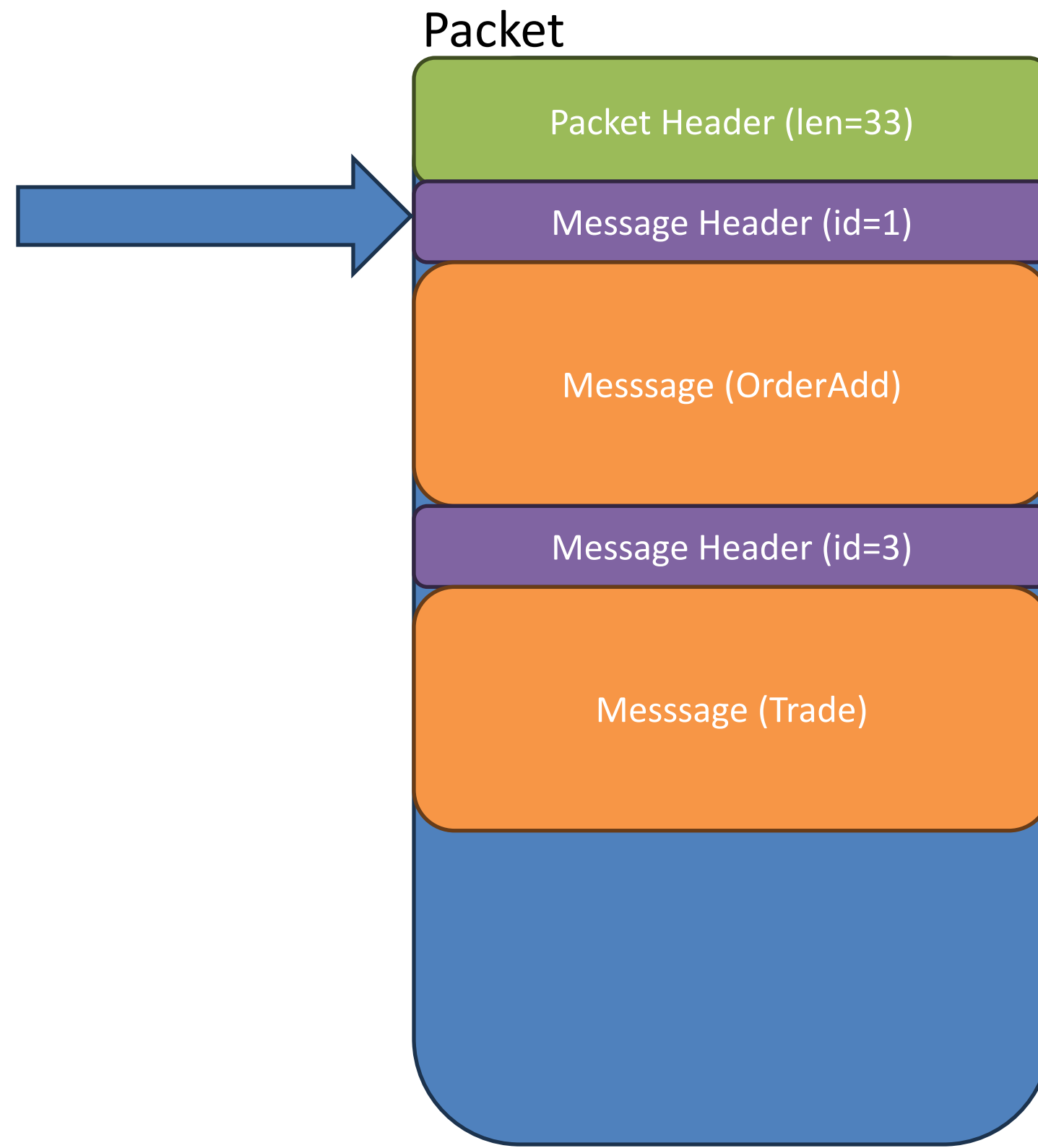
    auto packet_header_span = buffer.subspan(0, meta::size_of(packet_header_data_info));
    // data -> storage
    auto packet_header_storage = parse_message<typename [:packet_header_data_info:],
        typename [:packet_header_storage_info:]>(packet_header_span);
    auto payload_span = buffer.subspan(meta::size_of(packet_header_data_info),
        packet_header_storage.data_length);

    // let handler see the header
    handler.handle(std::move(packet_header_storage));
    ...
}
```

Finding the pieces: structs by name

```
constexpr meta::info
find_struct(meta::info ns, std::string_view name)
{
    auto ctx = meta::access_context::unchecked();
    for (meta::info m : meta::members_of(ns, ctx))
        if (meta::is_class_type(m) &&
            meta::has_identifier(m) &&
            meta::identifier_of(m) == name)
            return m;
    return meta::info{};
}
```

Multicast Packet example



parse_buffer: loop over messages, extract message header

```
void parse_buffer(std::span<std::byte const> buffer)
{
    ...
    constexpr auto message_header_data_info = find_struct(DataNs, "MessageHeader");
    constexpr auto message_header_storage_info = find_struct(StorageNs, "MessageHeader");

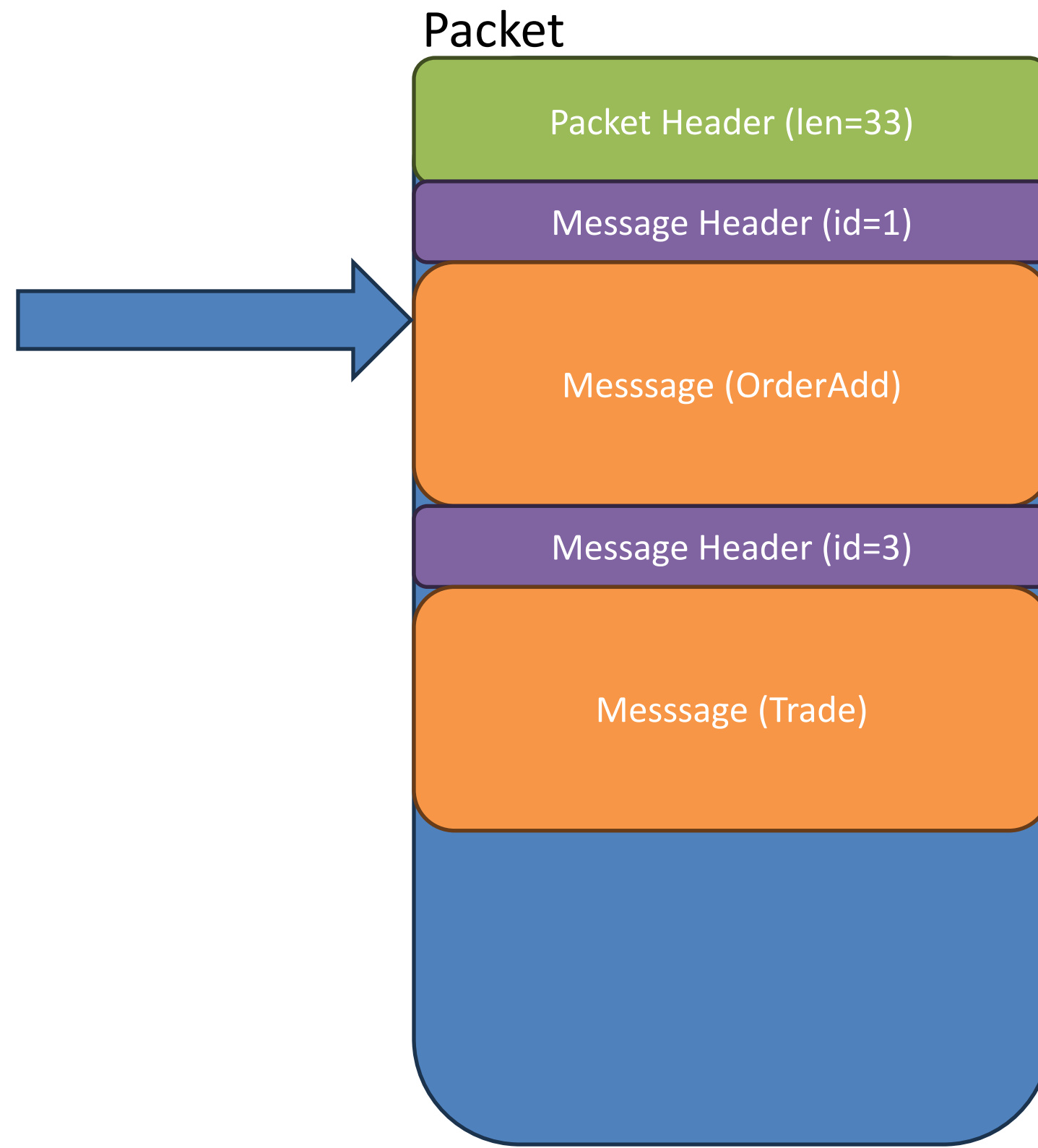
    while(payload_span.size() != 0)
    {
        // data -> storage
        auto message_header_storage = parse_message<typename [:message_header_data_info:],
            typename [:message_header_storage_info:]>(payload_span);

        auto current_message_type = message_header_storage.message_type;

        handler.handle(std::move(message_header_storage));

        ...
    }
}
```

Multicast Packet example



parse_buffer: extract matching message

```
void parse_buffer(std::span<std::byte const> buffer)
{
    ...
    template for(constexpr auto message_data_info: message_data_types)
    {
        constexpr auto message_storage_info = find_struct(StorageNs,
            meta::identifier_of(message_data_info));
        if(current_message_type == [:message_data_info:]::message_type_id)
        {
            if constexpr(message_storage_info != meta::info{}) // handling is optional
            {
                auto message_span = payload_span.subspan(meta::size_of(message_header_data_info));
                auto message_storage = parse_message<typename [:message_data_info:],
                    typename [:message_storage_info:]>(message_span);
                cb.handle(std::move(message_storage));
            }
            // next message
            payload_span = payload_span.subspan(meta::size_of(message_data_info));
            break;
        }
    }
}
```

Message fields with conversions

parse_message: looping over the data fields

```
template<typename MessageDataType, typename MessageStorageType>
    MessageStorageType parse_message(std::span<std::byte const> buffer)
{
    constexpr auto ctx = meta::access_context::unchecked();

    MessageStorageType message_storage;

    template for(constexpr auto data_field_info:
        std::define_static_array(meta::nonstatic_data_members_of(^MessageDataType, ctx)))
    {
        ...
    }

    return message_storage;
}
```

parse_message: find storage and extract

```
template<typename MessageDataType, typename MessageStorageType>
    MessageStorageType parse_message(std::span<std::byte const> buffer)
{
    ... // template for NSDM of data
    constexpr auto storage_field_info = find_nonstatic_member(^MessageStorageType,
        meta::identifier_of(data_field_info));

    constexpr auto field_data_size = meta::size_of(data_field_info);
    if constexpr(storage_field_info != meta::info{}) // handling is optional
    {
        auto field_data = get_data_field<typename [:meta::type_of(data_field_info):]>(
            buffer.subspan(0, field_data_size));
        // data -> storage
        copy_protocol_field(message_storage[:storage_field_info:], field_data);
    }
    // next field
    buffer = buffer.subspan(field_data_size);
    ...
}
```

get_data_field: actually grabbing the bytes

```
template<typename FieldDataType>
FieldDataType get_data_field(std::span<std::byte const> field_bytes)
{
    FieldDataType field_data;
    std::memcpy(&field_data, field_bytes.data(), field_bytes.size());
    return field_data;
}
```

Conversions are just an overload set

```
// generic: anything assignable
template<typename To, typename From>
    requires std::assignable_from<To&, From>
void copy_protocol_field(To& to, From from) { to = from; }

template<class T>
concept enumeration = std::is_enum_v<T>;
// enum <- integral
template<enumeration Enum, std::integral Num>
void copy_protocol_field(Enum& to, Num from)
{ to = static_cast<Enum>(from); }

// std::string <- fixed char array (null-terminator aware)
template<std::size_t N>
void copy_protocol_field(std::string& to, std::array<char, N> const& from)
{
    auto terminator = std::ranges::find(from, '\0');
    to.assign(from.data(), terminator == from.end() ?
              N : terminator - from.begin());
}
```

Extending it: a custom type, found by ADL

```
class StockId {
public:
    void set_raw_id(int id) { stock_id_ = id; }
    int get_raw_id() const { return stock_id_; }
private:
    int stock_id_ = 0;
};

// a plain free function in the same namespace as StockId
void copy_protocol_field(StockId& to, int from)
{
    to.set_raw_id(from);
}
```

Annotations

Annotations basics

- Annotations look similar to attributes but are distinct
- Attributes are about communicating with the compiler
 - `[[nodiscard]]`
 - `[[deprecated("favor API foo")]]`
- Annotations are about communicating with user code
 - `[[=MyAnnotation]]`
 - Serialization, command line parsing

Using annotations

```
struct [[=MyTypeAnnotation]] S
{
    [[=MyDataMemberAnnotation]] int foo;
};

[[=MyFunctionAnnotation]] int Bar();
```

- Annotations are indicated by the "=" sigil
- Must be a constant-expression with structural type
 - structural = allowed as non-type template parameters
- Applicable to
 - type, type alias, variable, function, namespace, enumerator, base-specifier, or non-static data member

Annotations as metadata

- What inherent properties of the protocol are informative to the parser?
 - Precision
 - Endianness
 - Semantics
- What properties of the storage types are informative?
 - Custom parsing
 - Non-trivial field mapping
- Types and support supplied by the protocol generator

Integral precision as annotation

```
struct FixedPrecision
{
    constexpr FixedPrecision(size_t p) : precision{p} {}
    size_t precision;
};

struct OrderAdd : MessageHeader
{
    static constexpr MessageTypeId message_type_id =
        MessageTypeId::order_add_message;
    uint32_t stock_id;
    uint32_t order_id;
    uint8_t side;
    uint32_t quantity;
    // 123456 -> 12.3456
    [[=protocol_generator::FixedPrecision(4)]] uint32_t price;
};
```

Reading an annotation's value off a field

```
constexpr std::optional<int> get_field_fixed_precision(meta::info field_data_info)
{
    for(auto a: meta::annotations_of_with_type(field_data_info, ^^FixedPrecision))
    {
        // can't splice param directly, 'a' is not a constant expression
        return meta::extract<FixedPrecision>(a).precision;
    }
    return std::nullopt;
}

template<typename MessageDataType, typename MessageStorageType>
MessageStorageType parse_message(std::span<std::byte const> buffer)
{
    ...
    if constexpr(constexpr auto precision = get_field_fixed_precision(data_field_info); precision)
        copy_protocol_field_with_precision<precision>(
            message_storage[:storage_field_info:], field_data);
    else
        copy_protocol_field(message_storage[:storage_field_info:], field_data);
    ...
}
```

Fixed precision price and handling

```
template<size_t PrecisionParam>
struct Price
{
    static constexpr size_t Precision = PrecisionParam;
    constexpr Price() = default;
    constexpr Price(int raw_price_param): raw_price{raw_price_param} {}
    template<size_t OtherPrecision>
    constexpr Price(Price<OtherPrecision> other):
        raw_price{other.raw_price * static_cast<int>(std::pow(10, Precision - OtherPrecision))}
    { static_assert(Precision >= OtherPrecision); }

    int raw_price = 0;
};

template<size_t DataPrecision, std::integral From, size_t StoragePrecision>
void copy_protocol_field_with_precision(Price<StoragePrecision>& to, From from)
{
    to = Price<StoragePrecision>{ Price<DataPrecision>{from} };
}
```

Protocol properties as annotation: NetworkByteOrder

```
struct NetworkByteOrderType {};  
static constexpr NetworkByteOrderType NetworkByteOrder;  
  
struct [[=protocol_generator::NetworkByteOrder]] OrderAdd : MessageHeader { ... };  
  
constexpr bool uses_network_byte_order(meta::info msg)  
{  
    for (auto a : meta::annotations_of_with_type(msg, ^^NetworkByteOrderType))  
        return true;  
    return false;  
}
```

Parser using NetworkByteOrder

```
template<typename FieldDataType, bool NetworkByteOrder>
FieldDataType get_data_field(std::span<std::byte const> field_bytes)
{
    FieldDataType field_data;
    std::memcpy(&field_data, field_bytes.data(), field_bytes.size());

    if constexpr(NetworkByteOrder)
        swap_byte_order(field_data);

    return field_data;
}
```

Custom Converter Per Field

```
template<typename Fn>
struct CustomConverter
{
    constexpr CustomConverter(Fn fnParam) : fn{fnParam} {}
    Fn fn;
};

void MySpecialConverter(StockId& to, int from)
{
    to.set_raw_id(from);
}

struct OrderAdd
{
    ...
    [[=protocol_generator::CustomConverter{&MySpecialConverter}]] StockId stock_id;
    ...
}
```

Finding the Custom Converter

```
constexpr meta::info get_custom_converter(meta::info field_data_info)
{
    for(auto a: meta::annotations_of(field_data_info))
    {
        auto converter_type = type_of(a);
        if(has_template_arguments(converter_type) &&
            template_of(converter_type) == ^^CustomConverter)
            return a;
    }
    return meta::info{};
}
```

Custom Converter Handling

```
template<typename MessageType, typename MessageStorageType>
MessageStorageType parse_message(std::span<std::byte const> buffer)
{
    ...
    if constexpr(constexpr auto custom_converter_info =
        get_custom_converter(storage_field_info); custom_converter_info != meta::info{})
    {
        constexpr auto custom_converter = [:meta::constant_of(custom_converter_info):];
        custom_converter.fn(message_storage[:storage_field_info:], field_data);
    }
    ...
}
```

Custom Converter With Parameters

```
struct StockRegistry
{
    StockId lookup_by_stock_id(int stock_id_num) const;
};

void MyConversionWithRegistry(StockId& to, int from, StockRegistry const& registry)
{
    to = registry.lookup_by_stock_id(from);
}

struct OrderAdd
{
    ...
    [[=protocol_generator::MakeConverter<StockRegistry&>(&MyConversionWithRegistry)]]
    StockId stock_id;
    ...
}

auto parser = protocol_generator::get_protocol_parser<^^OrderProtocol>(cb, stock_registry);
```

Custom Converter With Parameters

```
template<typename T, typename Fn>
struct CustomConverter
{
    using ParamType = T;
    constexpr CustomConverter(Fn fnParam)
        : fn{fnParam} {}
    Fn fn;
};

template<typename T, typename Fn>
constexpr CustomConverter<T, Fn> MakeConverter(Fn fn)
{
    return CustomConverter<T, Fn>{fn};
}

template<typename Fn>
constexpr CustomConverter<void, Fn> MakeConverter(Fn fn)
{
    return CustomConverter<void, Fn>{fn};
}
```

Keeping Converter Parameters

```
template<meta::info ProtocolNs, typename CallbackObject, typename... Ts>
struct Parser
{
    CallbackObject& cb;
    std::tuple<Ts...> converter_params;

    Parser(CallbackObject& cbParam, Ts&&... ts)
        : cb{cbParam}
        , converter_params{std::forward<Ts...>(ts...)} {}
    ...
}

template<meta::info ProtocolNs, typename CallbackObject, typename... Ts>
auto get_protocol_parser(CallbackObject& cb, Ts&&... ts)
{
    return Parser<ProtocolNs, CallbackObject, Ts...>{ cb, std::forward<Ts...>(ts...) };
}
```

Custom Converter With Parameters Handling

```
template<typename MessageType, typename MessageStorageType>
MessageStorageType parse_message(std::span<std::byte const> buffer)
{
    ...
    if constexpr(constexpr auto custom_converter_info =
        get_custom_converter(storage_field_info); custom_converter_info != meta::info{})
    {
        constexpr auto custom_converter = [:meta::constant_of(custom_converter_info):];
        using ParamType = typename [:std::meta::type_of(custom_converter_info):]:ParamType;
        if constexpr(^ParamType != ^void)
            custom_converter.fn(message_storage[:storage_field_info:], field_data,
                                std::get<ParamType>(converter_params));
        else
            custom_converter.fn(message_storage[:storage_field_info:], field_data);
    }
}
...
```

Explicit Data Field Source

```
template<std::meta::info FieldInfo>
struct FieldReference
{
    static constexpr auto field_info = FieldInfo;
};

struct OrderAdd
{
    ...
    [[=protocol_generator::FieldReference<^^OrderProtocol::OrderAdd::order_id>{}]]
    int add_order_id;
    ...
}
```

Parser Generator Interface Choices

- Hard-coded protocol definition requirements
 - Enclosing namespace with "Data" and "Storage" sub-namespaces
 - struct "PacketHeader"
 - "data_length" member
 - struct "MessageHeader"
 - "message_type" member
- Message structures with static "message_type_id" matching "message_type"

Fully Annotated Protocol Definition

```
#pragma pack(push, 1)
struct [[=protocol_generator::NetworkByteOrder]] [[=protocol_generator::PacketHeader]]
PacketHeader
{
    [[=protocol_generator::PacketHeaderDataLength]] std::uint16_t data_length;
    std::uint32_t sequence_number;
};

struct [[=protocol_generator::NetworkByteOrder]] [[=protocol_generator::MessageHeader]]
MessageHeader
{
    [[=protocol_generator::MessageId]] std::uint8_t message_type;
};
```

Fully Annotated Protocol Definition

```
struct [[=protocol_generator::NetworkByteOrder]] OrderAdd: public MessageHeader
{
    [[=protocol_generator::MessageId]] static constexpr MessageTypeId message_type_id
        = MessageTypeId::order_add_message;

    std::uint32_t stock_id;
    std::uint32_t order_id;
    std::uint8_t side;
    std::uint32_t quantity;
    [[=protocol_generator::FixedPrecision(4)]] std::uint32_t price;
};
```

Fully Annotated Protocol Definition - Comparison

```
void parse_buffer(std::span<std::byte const> buffer)
{
    ...
    constexpr auto packet_header_data_info = find_struct(DataNs, "PacketHeader");
    constexpr auto packet_header_storage_info = find_struct(StorageNs, "PacketHeader");
    ...
}
```

```
void parse_buffer(std::span<std::byte const> buffer)
{
    ...
    constexpr auto packet_header_data_info =
        find_member_with_annotation(DataNs, ^^PacketHeaderType);
    constexpr auto packet_header_storage_info =
        find_struct(StorageNs, meta::identifier_of(packet_header_data_info));
    ...
}
```

Finding annotations

```
consteval meta::info find_member_with_annotation(meta::info ns,
    meta::info annotation)
{
    auto ctx = meta::access_context::unchecked();
    for (meta::info m : meta::members_of(ns, ctx))
    {
        if(meta::annotations_of_with_type(m, annotation).size())
            return m;
    }
    return meta::info{};
}
```

Fully Annotated Protocol Definition

```
template<meta::info DataNs, meta::info StorageNs, typename CallbackObject, typename... Ts>
struct Parser
{
void parse_buffer(std::byte const const buffer)
{
...
constexpr auto packet_header_data_info =
    find_member_with_annotation(DataNs, ^^PacketHeaderType);
constexpr auto packet_header_storage_info =
    find_struct(StorageNs, meta::identifier_of(packet_header_data_info));
...
constexpr auto packet_header_data_length_info =
    find_member_with_annotation(packet_header_data_info, ^^PacketHeaderDataLengthType);
constexpr auto packet_storage_data_length_info =
    find_nonstatic_member(packet_header_storage_info,
        meta::identifier_of(packet_header_data_length_info));

auto data_length = packet_header_storage.[:packet_storage_data_length_info:];
```

Fully Annotated Protocol Definition

```
void parse_buffer(std::span<std::byte const> buffer)
{
    ...
    constexpr auto message_header_data_info =
        find_member_with_annotation(DataNs, ^^MessageHeaderType);
    constexpr auto message_header_storage_info = find_struct(StorageNs,
        meta::identifier_of(message_header_data_info));
    ...
    constexpr auto message_header_data_message_id_info =
        find_member_with_annotation(message_header_data_info, ^^MessageIdType);
    constexpr auto message_header_storage_message_id_info =
        find_nonstatic_member(message_header_storage_info,
            meta::identifier_of(message_header_data_message_id_info));
    ...
    template for(constexpr auto message_data_info: message_data_members)
    {
        constexpr auto message_storage_info = find_struct(StorageNs,
            meta::identifier_of(message_data_info));
        constexpr auto message_id_info = find_member_with_annotation(message_data_info, ^^MessageIdType);
        if(message_header_storage.[:message_header_storage_message_id_info:] == [:message_id_info:])
        {
            ...
        }
    }
}
```

Conclusion

Reflections on reflection for protocol generation

- Compile time programming and its caveats
 - Structural types
 - Only transitive compile time allocation
 - Constant expressions
 - Immediate functions
 - Debugging difficult (CLion constexpr debugger)
- Reflection capabilities
 - Very flexible
 - Access everything via free functions
 - Sharp corners can catch you

Reflections on reflection for protocol generation

- Protocol Handling
 - Basics appear easy once patterns are apparent
 - Annotations very rich
 - Arbitrarily capable compile time DSL, declarative
- Enhancements
 - Variable length fields with reference to count field
 - Provide handler both aliasing pointer and type converted rider
 - Sequence number handling
 - Timestamp generation
 - Order book construction, join field annotations

Acknowledgements

Questions?